



```
In [1]: import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
import warnings
warnings.filterwarnings('ignore')

# Set random seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

```
In [2]: def load_fashion_mnist():
    """Load and preprocess the Fashion MNIST dataset."""
    print("Loading Fashion MNIST dataset...")
    (X_train, y_train), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

    # Normalize pixel values to range [0, 1]
    X_train = X_train.astype('float32') / 255.0
    X_test = X_test.astype('float32') / 255.0

    # Reshape images to include channel dimension
    X_train = X_train.reshape(-1, 28, 28, 1)
    X_test = X_test.reshape(-1, 28, 28, 1)

    # Convert labels to categorical
    y_train = tf.keras.utils.to_categorical(y_train, 10)
    y_test = tf.keras.utils.to_categorical(y_test, 10)

    return (X_train, y_train), (X_test, y_test)

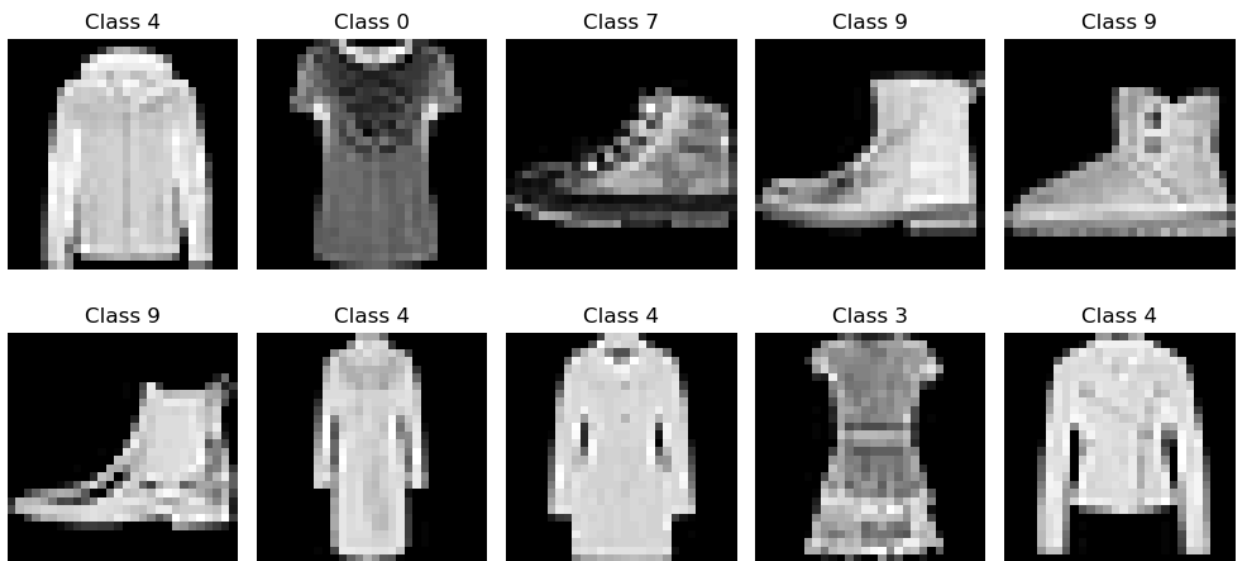
# Load and preprocess data
(X_train, y_train), (X_test, y_test) = load_fashion_mnist()

# Create validation split
val_samples = 5000
X_val = X_train[:val_samples]
y_val = y_train[:val_samples]
X_train = X_train[val_samples:]
y_train = y_train[val_samples:]

print(f"Training set shape: {X_train.shape}")
print(f"Validation set shape: {X_val.shape}")
print(f"Test set shape: {X_test.shape}")

# Display some sample images
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i].reshape(28, 28), cmap='gray')
    plt.title(f'Class {np.argmax(y_train[i])}')
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Loading Fashion MNIST dataset...
 Training set shape: (55000, 28, 28, 1)
 Validation set shape: (5000, 28, 28, 1)
 Test set shape: (10000, 28, 28, 1)



```
In [3]: def create_cnn_model():
  """Creates a CNN model for Fashion MNIST classification."""
  model = models.Sequential([
    # First Convolutional Block
    layers.Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(28, 28, 1)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Second Convolutional Block
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Third Convolutional Block
    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Dense Layers
    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
```

```
        layers.Dense(10, activation='softmax')
    ])

    return model

# Create and compile model
model = create_cnn_model()
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None , 28, 28, 32)	
batch_normalization (BatchNormalization)	(None , 28, 28, 32)	
conv2d_1 (Conv2D)	(None , 28, 28, 32)	
batch_normalization_1 (BatchNormalization)	(None , 28, 28, 32)	
max_pooling2d (MaxPooling2D)	(None , 14, 14, 32)	
dropout (Dropout)	(None , 14, 14, 32)	
conv2d_2 (Conv2D)	(None , 14, 14, 64)	
batch_normalization_2 (BatchNormalization)	(None , 14, 14, 64)	
conv2d_3 (Conv2D)	(None , 14, 14, 64)	
batch_normalization_3 (BatchNormalization)	(None , 14, 14, 64)	
max_pooling2d_1 (MaxPooling2D)	(None , 7, 7, 64)	
dropout_1 (Dropout)	(None , 7, 7, 64)	
conv2d_4 (Conv2D)	(None , 7, 7, 128)	
batch_normalization_4 (BatchNormalization)	(None , 7, 7, 128)	
conv2d_5 (Conv2D)	(None , 7, 7, 128)	
batch_normalization_5 (BatchNormalization)	(None , 7, 7, 128)	
max_pooling2d_2 (MaxPooling2D)	(None , 3, 3, 128)	
dropout_2 (Dropout)	(None , 3, 3, 128)	
flatten (Flatten)	(None , 1152)	
dense (Dense)	(None , 512)	
batch_normalization_6 (BatchNormalization)	(None , 512)	
dropout_3 (Dropout)	(None , 512)	
dense_1 (Dense)	(None , 10)	

Total params: 885,738 (3.38 MB)

Trainable params: 883,818 (3.37 MB)

Non-trainable params: 1,920 (7.50 KB)

```
In [4]: # Training parameters
EPOCHS = 10
BATCH_SIZE = 64

# Callbacks
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=3
    )
]

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(X_val, y_val),
    callbacks=callbacks,
    verbose=1
)
```

Epoch 1/10
860/860 ————— **105s** 115ms/step - accuracy: 0.8099 - loss: 0.5479
- val_accuracy: 0.8952 - val_loss: 0.2845 - learning_rate: 0.0010

Epoch 2/10
860/860 ————— **101s** 118ms/step - accuracy: 0.8789 - loss: 0.3301
- val_accuracy: 0.9094 - val_loss: 0.2450 - learning_rate: 0.0010

Epoch 3/10
860/860 ————— **100s** 116ms/step - accuracy: 0.8988 - loss: 0.2815
- val_accuracy: 0.9162 - val_loss: 0.2310 - learning_rate: 0.0010

Epoch 4/10
860/860 ————— **100s** 116ms/step - accuracy: 0.9057 - loss: 0.2598
- val_accuracy: 0.9138 - val_loss: 0.2390 - learning_rate: 0.0010

Epoch 5/10
860/860 ————— **101s** 117ms/step - accuracy: 0.9150 - loss: 0.2370
- val_accuracy: 0.9254 - val_loss: 0.1988 - learning_rate: 0.0010

Epoch 6/10
860/860 ————— **100s** 116ms/step - accuracy: 0.9188 - loss: 0.2231
- val_accuracy: 0.9218 - val_loss: 0.1985 - learning_rate: 0.0010

Epoch 7/10
860/860 ————— **101s** 117ms/step - accuracy: 0.9230 - loss: 0.2098
- val_accuracy: 0.9216 - val_loss: 0.2144 - learning_rate: 0.0010

Epoch 8/10
860/860 ————— **109s** 126ms/step - accuracy: 0.9273 - loss: 0.2000
- val_accuracy: 0.9276 - val_loss: 0.1942 - learning_rate: 0.0010

Epoch 9/10
860/860 ————— **117s** 136ms/step - accuracy: 0.9302 - loss: 0.1929
- val_accuracy: 0.9092 - val_loss: 0.2421 - learning_rate: 0.0010

Epoch 10/10
860/860 ————— **131s** 153ms/step - accuracy: 0.9353 - loss: 0.1759
- val_accuracy: 0.9186 - val_loss: 0.2238 - learning_rate: 0.0010

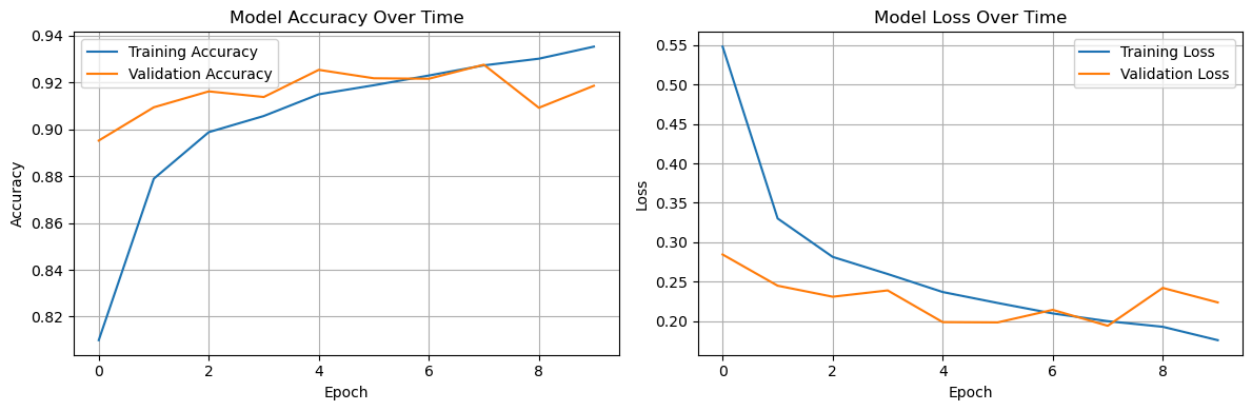
```
In [5]: def plot_training_history(history):
        """Plot the training and validation accuracy/loss curves."""
        plt.figure(figsize=(12, 4))

        # Accuracy plot
        plt.subplot(1, 2, 1)
        plt.plot(history.history['accuracy'], label='Training Accuracy')
        plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
        plt.title('Model Accuracy Over Time')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.legend()
        plt.grid(True)

        # Loss plot
        plt.subplot(1, 2, 2)
        plt.plot(history.history['loss'], label='Training Loss')
        plt.plot(history.history['val_loss'], label='Validation Loss')
        plt.title('Model Loss Over Time')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()
        plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

```
plot_training_history(history)
```



```
In [7]: def plot_sample_predictions(model, X_test, y_test, num_samples=10):
        """Plot sample predictions with actual and predicted labels."""
        class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
                        'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

        predictions = model.predict(X_test[:num_samples])

        plt.figure(figsize=(15, 4))
        for i in range(num_samples):
            plt.subplot(2, 5, i + 1)
            plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
            true_label = class_names[np.argmax(y_test[i])]
            pred_label = class_names[np.argmax(predictions[i])]
            color = 'green' if true_label == pred_label else 'red'
            plt.title(f'True: {true_label}\nPred: {pred_label}', color=color)
            plt.axis('off')
        plt.tight_layout()
        plt.show()

        # Evaluate on test set
        test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
        print(f"\nTest Accuracy: {test_accuracy:.4f}")

        # Plot sample predictions
        print("\nGenerating sample predictions...")
        plot_sample_predictions(model, X_test, y_test)

        # Plot confusion matrix
        from sklearn.metrics import confusion_matrix
        import seaborn as sns

        y_pred = np.argmax(model.predict(X_test), axis=1)
        y_true = np.argmax(y_test, axis=1)
        cm = confusion_matrix(y_true, y_pred)
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
```

Test Accuracy: 0.9207

Generating sample predictions...

1/1 ————— 0s 112ms/step

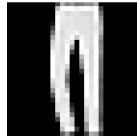
True: Ankle boot
Pred: Ankle boot



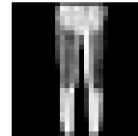
True: Pullover
Pred: Pullover



True: Trouser
Pred: Trouser



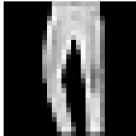
True: Trouser
Pred: Trouser



True: Shirt
Pred: Shirt



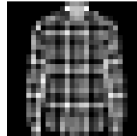
True: Trouser
Pred: Trouser



True: Coat
Pred: Coat



True: Shirt
Pred: Shirt



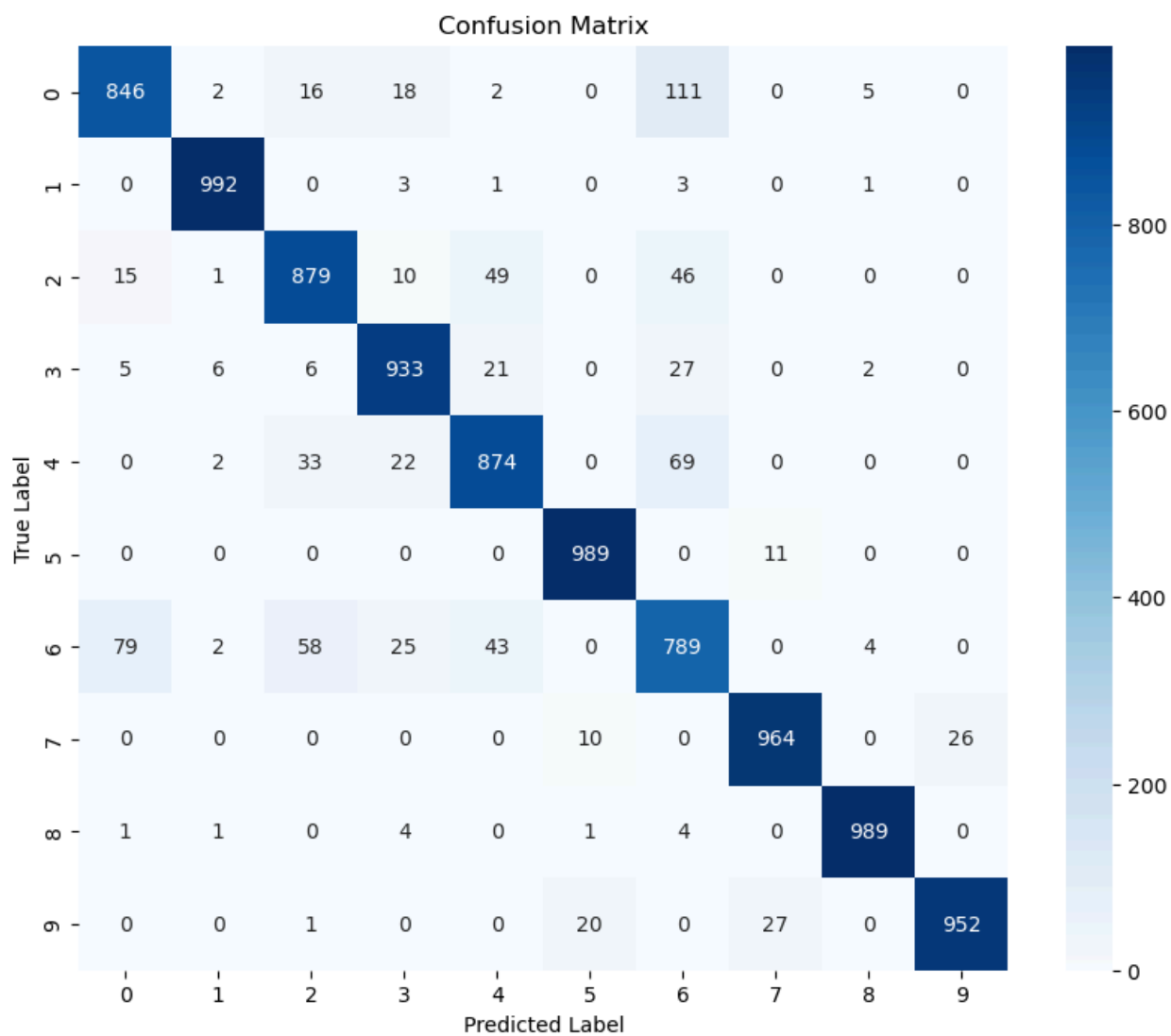
True: Sandal
Pred: Sandal



True: Sneaker
Pred: Sneaker



313/313 ————— 8s 24ms/step



In []: